# Job Application Agent — Problem Statement & Build Brief (Definitive)

*This supersedes all earlier briefs. It is written as a problem statement: context first, then what must exist, the constraints it must satisfy, the decisions already made, the decisions left to you, and how "done" is judged. Build in the stage order given.*

---

## 1. Context

A job-seeker applies to a **fixed, known set of role families** (e.g. backend, data, ML-adjacent — you will define the exact list). Two problems make the search slow and low-yield:

- Generic resumes get filtered out by ATS keyword matching; tailoring each one by hand doesn't scale.
- Job boards surface stale, irrelevant, and duplicate postings, so time is wasted reading before applying.

The project must solve this for **one real user (you)** while doubling as a **portfolio artifact**. Both goals are load-bearing: a tool that isn't genuinely useful produces a hollow portfolio piece, and a useful tool that a reviewer can't understand or trust in three minutes wastes the portfolio value. Where the two goals compete, **honesty and judgment win over impressiveness** — an accurately described modest system beats an over-claimed clever one.

---

## 2. The problem statement (one paragraph)

> Build and deploy a scheduled backend service that, given a target **role**, discovers fresh job postings from a sanctioned source, uses a language model to **decide which postings are worth pursuing for that specific role**, tailors the user's resume to each worthwhile posting without fabricating anything, and delivers a **deduplicated** daily email digest with a per-job relevance score, reason, and tailored-resume link. The service must run unattended on a schedule, support manual on-demand runs with a caller-supplied role, never re-process a posting it has seen before, and be deployed, CI-tested, and documented well enough that a stranger understands the architecture and one non-trivial design decision quickly.

---

## 3. Domain & positioning (how to describe it)

- **Primary domain:** backend / API engineering — this is what the code overwhelmingly *is*.
- **Secondary domain:** applied LLM / AI-application development — integrating a model into a real workflow with prompt design and output constraints.
- **Also touches:** systems integration, automation, light DevOps (deploy + scheduled CI).
- **It is NOT:** data science, ML engineering (no training/data pipelines), distributed systems, frontend, or an autonomous agent. Do not claim these — a reviewer will empty the claim out.

The same repo is pitched to a backend team as "a clean Python service with real third-party integration" and to an AI-product team as "someone who wires an LLM into production with cost and safety awareness." Same code, different first sentence depending on the role you target.

---

## 4. Current state (starting point)

**Works end-to-end via `POST /run`:** layered FastAPI app (routes → services → integrations → models); concurrent two-source job fetch with graceful per-source failure; Google-Doc→text resume read; LLM rewrite with an anti-fabrication guardrail; Gmail digest. Secrets gitignored. Architecture is clean.

**Broken / must fix:**

- `requirements.txt` is UTF-16 — regenerate as UTF-8.
- `resume_reader.py` hardcodes a Windows path; must use `settings.service_account_file`.
- `tests/output.py` isn't collected by pytest (wrong filename) and contains an assertion that contradicts the implementation — rename and fix, or the test suite is dead and dishonest.
- `doc_creator` is commented out of the pipeline, so the digest dumps full resume text inline instead of linking a doc — decide on or off and make the email consistent.

**Missing (the substance of this brief):** role-aware scoring, cross-run deduplication, per-run role input, scheduling, deployment, CI, README.

---

## 5. Functional requirements

| # | Capability | Requirement |
|---|---|---|
| F1 | **Role input** | Manual run accepts a caller-supplied role; scheduled run falls back to a configured default. Role is a **constrained enum**, not free text (see §6). |
| F2 | **Discovery** | Fetch recent postings for the role from a **sanctioned API** (Adzuna). Return structured `Job` objects. Keep the dead Indeed-RSS path only as an honestly-commented demonstration of multi-source design. |
| F3 | **Persistent dedup** | Never process a posting seen in a prior run. State survives restarts (SQLite — stdlib, zero new dependency). |
| F4 | **Resume source** | Load the user's master resume as text. **Open decision:** one resume, or one profile per role family (see §7). |
| F5 | **Role-aware scoring** | For each job, the LLM returns a 0–10 relevance score + one-line reason, judged against a **role-specific rubric**. Only jobs at or above a threshold proceed to rewrite. |
| F6 | **Tailoring** | LLM rewrites the resume to surface *real* matching keywords. Hard constraint: never invent facts, metrics, employers, dates, or skills. |
| F7 | **Output** | Save each tailored resume (Google Doc) and produce a shareable link. |
| F8 | **Digest** | One email; per job: company, role, apply link, **score, reason**, tailored-resume link; grouped/readable. |

## 6. The role dimension (design spine — read carefully)

"Role" is not a single input; it threads through **three** places, and they must all key off **one shared role value** to stay coherent:

1. **Input** (F1) — the caller picks a role at manual run; the scheduler uses a default.
2. **Resume selection** (F4) — which resume/profile represents the candidate for that role.
3. **Scoring rubric** (F5) — what "a strong match" means for that role.

**Design rule:** one enum value flows into all three as a dictionary key. Concretely:

- Role is a Pydantic **enum** (`Literal[...]` / `Enum`), so invalid roles are rejected automatically, the supported set is self-documenting, and every valid role is

guaranteed to have a rubric (and profile, if you add them).

- **Do not write N separate scoring prompts.** Use **one prompt template with a `{role}` and `{role_rubric}` slot**, plus a small `dict[role, rubric_string]`. The invariant instructions (scale definition, output format, "return only a number and one line") live once; only the rubric varies. Duplicated prompts drift and signal copy-paste; a parameterized template signals abstraction.

Because your role set is **fixed and known**, constraining the input is strictly better than free text — free text forces fuzzy string-to-key mapping, a problem you only have if you fail to constrain the input.

---

## 7. Open decisions (you must choose — don't relitigate the rest)

- **D1 — Per-role resume profiles?** One master resume can't represent genuinely different role families well; a scoring rubric can't fix a resume that's 70% wrong for the role. If your roles diverge, F4 becomes a small set of profiles keyed by the *same* role enum. Decide before Stage 2, because it changes F4's shape.
- **D2 — Score threshold value.** Too high → empty digests; too low → wasted rewrites. Pick a sane default, make it overridable, tune after observing real scores.
- **D3 — The role set itself.** Enumerate the actual families you apply to (this defines the enum, the rubric dict, and — if D1 = yes — the profile set).

---

## 8. Decisions already made (do not re-open)

- **Input transport: request body, not query string.** `/run` takes a Pydantic `RunRequest` (role enum required or defaulted; location, threshold optional). Body = structured, extensible, not logged in URLs. Query strings are for a couple of simple read-filters; this is a growing parameter set on an action endpoint.
- **API over scraping.** Official Adzuna API, never LinkedIn scraping (ToS violation + brittle).
- **Human-in-the-loop.** The service prepares applications; the user reviews and submits. No auto-submit. This is a deliberate design choice, not a limitation.
- **Scheduler = GitHub Actions cron** hitting `/run`, not in-process APScheduler (doubles as CI/CD evidence; keeps a free instance from needing to stay warm).
- **Skip:** PDF/LaTeX rendering (a Doc link suffices) and adding a second *new* live source. Both are high-effort, low-portfolio-value here.
- **Not agentic.** It's a pipeline with one model-driven control-flow decision (the scoring gate). Describe it exactly that way.

---

## 9. Non-functional requirements / constraints

- **Cost model (state it honestly).** Scoring adds a *cheap* call (≈20 output tokens: a number + one line, on a truncated job description) to avoid an *expensive* rewrite (≈1,500–2,000 output tokens) on jobs that don't merit one. At the current low `jobs_per_run`, this is **not** a real money saving — it is a **quality filter now and a scaling guardrail at high volume.** Claim the benefit that's true for your actual N.
- **Security.** No secret in source or URL; least-privilege Google scopes; verify `.env` / `service_account.json` never entered git history.
- **Reliability.** One failing source or malformed job must not kill the run (already true — preserve it).
- **Idempotency.** Re-running a day produces no duplicate emails/docs (depends on F3).
- **Observability.** Structured logs answer "what did the scheduled run do, and why did each job pass or fail scoring."
- **Testability.** Dedup, score-parsing, and keyword logic unit-tested **without network**; tests **collected** and green in CI.

---

## 10. Build sequence (build in this order)

**Stage 0 — Unblock (~20 min).** Fix UTF-16 `requirements.txt`; fix hardcoded path; resolve `doc_creator` (on, or make the email honest about inline). *Done:* fresh `pip install -r` works; `/run` runs with no machine-specific path.

**Stage 1 — Role input + persistent dedup.** Add `RunRequest` (enum role, optional fields, config default for the scheduler); thread the role through `get_jobs`. Add SQLite dedup keyed on job URL. *Done:* a manual run with a chosen role works; a second run the same day processes zero already-seen jobs. *(Dedup precedes the scheduler — a daily run without it re-spams you, which is worse than no scheduler.)*

**Stage 2 — Role-aware scoring gate.** One prompt template + rubric dict keyed by the role enum; score each job; only ⩾ threshold get rewritten; digest shows score + reason. *Done:* sub-threshold jobs are visibly skipped, not rewritten, and scores reflect the role's rubric.

**Stage 3 — Scheduler.** GitHub Actions cron → `/run` with the default role. *Done:* an unattended run yields a correct, deduped, scored digest.

**Stage 4 — Legibility: deploy + CI + README.** Get a minimal deploy + green CI badge up early (right after Stage 1) so "it's live" is true; write the *real* README last, once it describes things that exist. *Done:* live URL, passing CI badge, README a stranger parses in < 3 min.

**Stage 5 (optional) — Concurrency.** Parallelize per-job rewrites with `asyncio.gather` (today they're a sequential loop). Only after this is a "concurrent LLM inference" claim true — and then measure the speedup, don't guess it.

---

## 11. Honesty & claim discipline (portfolio integrity)

The gap between what you *claim* and what the repo *verifies* is the whole game. Hold these lines:

- **"Agent" → "pipeline with a model-driven scoring gate."** Accurate and still impressive.
- **Scoring → "quality filter / scaling guardrail,"** not "cost savings," at current volume.
- **Concurrency claim → only after Stage 5,** and with a measured number.
- **"Testing" is not a claimable skill until the suite is collected and green.** Right now it isn't.
- **Every resume line must survive a reviewer opening the repo.** If the code can't back it, cut it.

---

## 12. Skills demonstrated (for the resume / interview)

**Verifiable in the repo now (or after the stages):** API & backend architecture (FastAPI, layered design, typed contracts); async Python & concurrency (`httpx`, `asyncio.gather`, `run_in_executor`); third-party integration (Adzuna, Google service-account auth + scopes, Groq, SMTP); graceful degradation (`return_exceptions`, partial success); config & security hygiene (`pydantic-settings`, `extra="forbid"`, gitignored secrets); applied LLM integration & prompt design (constrained system prompt, anti-fabrication guardrail, parameterized rubric); **after the stages:** idempotency/persistence design, cost-aware LLM gating, CI/CD & scheduled deployment.

**Lead with the meta-skill: engineering judgment** — API-over-scraping, human-in-the-loop, scoping out the shiny stuff, and describing the system's limits accurately (including dropping "agentic"). Interviewers test for exactly this and most candidates fail it.

**Resume line (true only after Stages 0–4; the metric only after Stage 5):**

> **AI Job-Application Agent** — *FastAPI · Groq (Llama 3.3) · Google APIs · SQLite · GitHub Actions · Render.* Built and deployed a scheduled backend service that discovers roles via the Adzuna API, applies a role-specific LLM relevance-scoring gate to decide which postings merit tailoring, rewrites the resume per role under a strict no-fabrication

> constraint, and emails a deduplicated daily digest. Designed for idempotent unattended runs with CI-tested core logic.

---

## 13. What a skeptic attacks (pre-empt in interview)

- *"That's just LinkedIn scraping — ToS violation."* → No; official Adzuna API, chosen for that reason.
- *"It's not really an agent."* → Correct; it's a pipeline with one model-driven decision. Never claimed otherwise.
- *"Scoring adds an LLM call — how is that cheaper?"* → It isn't, at low volume; it's a quality filter now and a scaling guardrail at high N. (Owning this nuance is the strongest possible answer.)
- *"Is it live or a demo?"* → Live URL, scheduled runs, idempotent dedup.
- *"What did you decide vs. copy?"* → Source choice, role-keyed design across three touchpoints, scoring gate, idempotency, input transport.

---

## 14. Definition of done

- [ ] Fresh `pip install -r` works; `/run` runs with no machine-specific path.
- [ ] `POST /run` accepts a valid role enum (rejects invalid); scheduler uses the default.
- [ ] Two runs in one day → zero duplicate jobs/emails (F3).
- [ ] Each digest job shows a role-appropriate score + reason; sub-threshold jobs skipped, not rewritten.
- [ ] Tailored resumes are factually consistent with the source (no invented content).
- [ ] One failing source still yields a digest.
- [ ] Role, location, threshold, jobs-per-run changeable without code edits.
- [ ] Tests collected and green in CI on every push.
- [ ] Live deployed URL; README explains architecture + one non-trivial decision in < 3 min.
- [ ] No claim anywhere (resume, README, interview) the repo can't back up.

---

*Build Stage 0 → 4 in order; decide D1–D3 before Stage 2. Bring the Stage 1 and Stage 2 code back for review against this spec — not just the plan.*